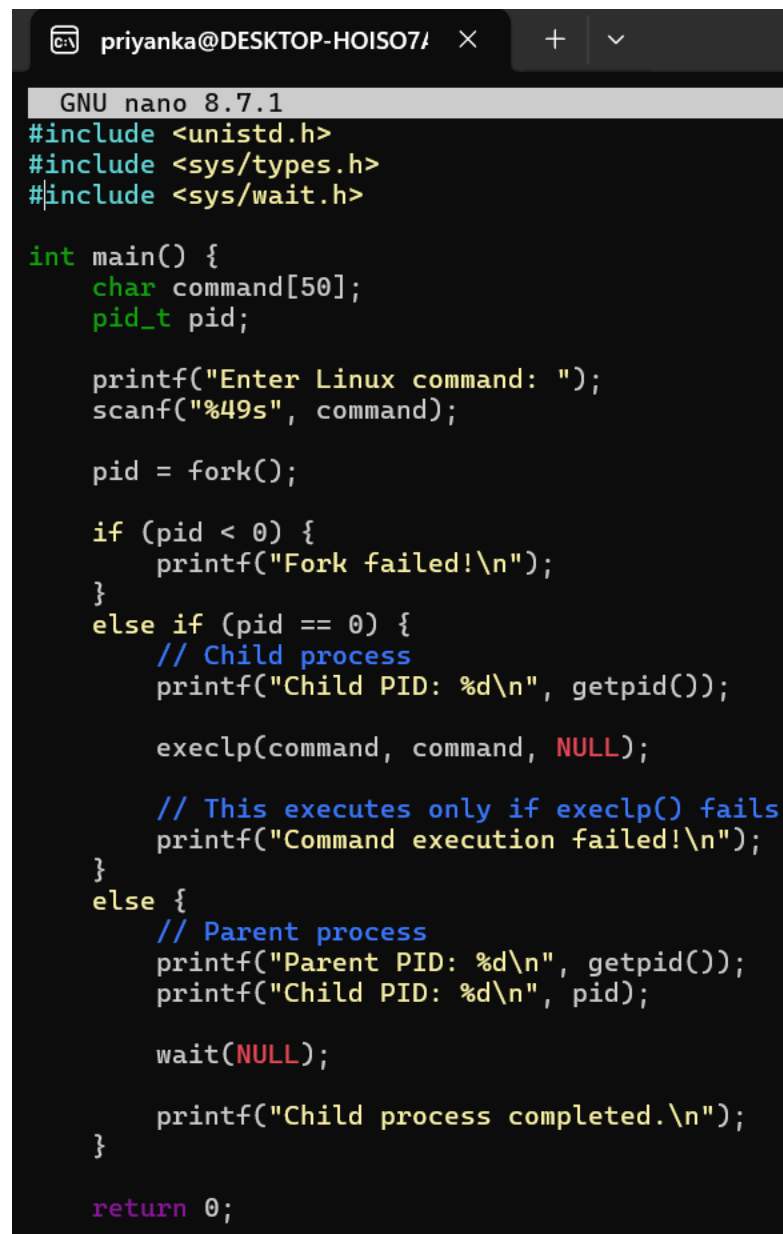


Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that

1. Accept a Linux command as input.
2. Create a child process using `fork()`.
3. Execute the command in the child process using an appropriate `exec()` system call.
4. Allow the parent process to wait for the child using `wait ()`.
5. Display the Process ID (PID) of both parent and child processes.

Code:

A screenshot of a terminal window with a dark background. The window title bar shows 'priyanka@DESKTOP-HOISO7/'. The terminal content shows the GNU nano 8.7.1 editor with a C program. The program includes headers for unistd.h, sys/types.h, and sys/wait.h. It defines a main function that prompts the user for a Linux command, reads it into a 50-character array, and then forks a child process. In the child process (pid == 0), it prints its PID and attempts to execute the command using execlp. If execlp fails, it prints an error message. In the parent process, it prints both its own PID and the child's PID, then calls wait(NULL) to wait for the child to complete, and finally prints a message indicating the child process has completed. The program returns 0 at the end.

```
GNU nano 8.7.1
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    char command[50];
    pid_t pid;

    printf("Enter Linux command: ");
    scanf("%49s", command);

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child PID: %d\n", getpid());

        execlp(command, command, NULL);

        // This executes only if execlp() fails
        printf("Command execution failed!\n");
    }
    else {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

### Output:

```
priyanka@DESKTOP-H0IS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practica
l/Week 1$ ls
task1.1.c
priyanka@DESKTOP-H0IS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practica
l/Week 1$ gcc task1.1.c -o task1.1
priyanka@DESKTOP-H0IS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practica
l/Week 1$ ls
task1.1 task1.1.c
priyanka@DESKTOP-H0IS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practica
l/Week 1$ ./task1.1
Enter Linux command: ls
Parent PID: 566
Child PID: 567
Child PID: 567
task1.1 task1.1.c
Child process completed.
priyanka@DESKTOP-H0IS07A:/mnt/c/Users/DELL/OneDrive/Documents/S OS/Practica
l/Week 1$ |
```